

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: "Python "

A efectuat :

st. gr.SI – 261, Vizitiu Laura

A verificat :

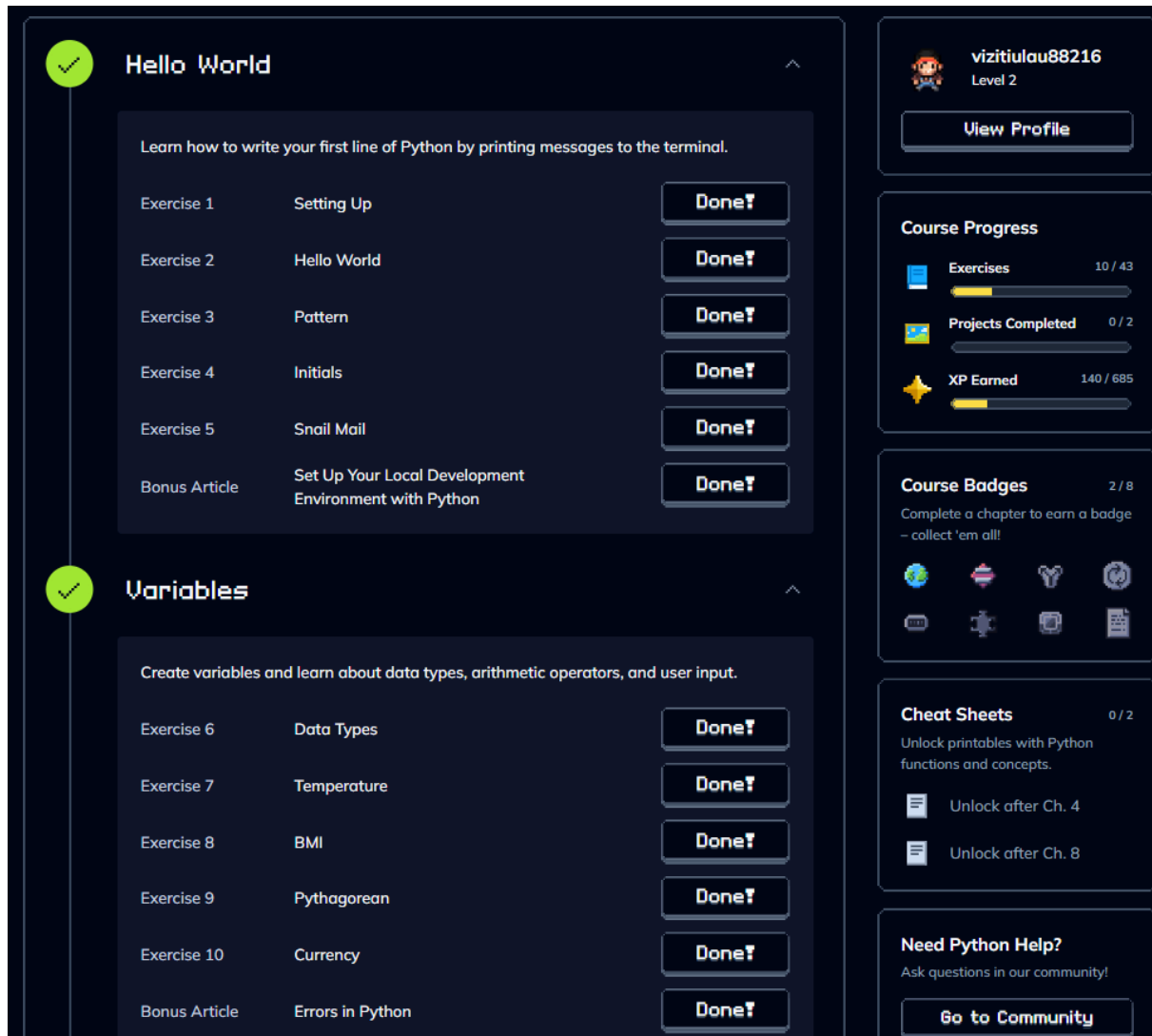
asistent universitar, Dumitrașcu Marius

Chișinău – 2026

Sarcina lucrării



Imaginea 1.1 Codedex



Imaginea 1.2 Sarcini realizate Codedex

Exercise

Use the "print" function to print the line "Hello, World!".

script.py

1 print("Hello, World!")

IPython Shell

<script.py> output:
Hello, World!

In [1]: |

Great job!

Solution

Submit

●

🔗

Imaginea 2.1 <https://www.learnpython.org/> Learn the Basics - Hello,World!

Exercise

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20. [Python number types](#)

script.py

1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8 print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10 print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12 print("Integer: %d" % myint)

IPython Shell

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]: |

Great job!

Solution

Submit

●

🔗

Imaginea 2.2 Learn the Basics - Variables and Types

Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable `second_name` with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py

```
1 # TODO: change this code
2
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 numbers.append(1)
7 numbers.append(2)
8 numbers.append(3)
9
10 strings.append("hello")
11 strings.append("world")
12 second_name = names[1]
13
14
15 # this code should write out the filled arrays
16 # and the second name in the names list (Eric).
17 print(numbers)
18 print(strings)
```

IPython Shell

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is Eric

In [1]:

Great Job!

Solution

Submit

Imaginea 2.3 Learn the Basics - Lists

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created. [Master Coding Skills](#)

script.py

```
3
4 # TODO: change this code
5 x_list = [x] * 10
6 y_list = [y] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len(big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) == 10:
15     print("Almost there...")
16 if big_list.count(x) == 10 and big_list.count(y) == 10:
```

IPython Shell

<script.py> output:
x_list contains 1 objects
y_list contains 1 objects
big_list contains 0 objects

<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
Almost there...
Great!

In [1]:

Good work!

Solution

Submit

Imaginea 2.4 Learn the Basics - Basic Operators

Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.` [Data format converter](#)

script.py

```
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance
  is %s."
3
4 print(format_string % data)
```

IPython Shell

```
<script.py> output:
Hello John Doe. Your current balance is $53.44.

In [1]:
```

Great work!

Solution

Submit

Imaginea 2.5 Learn the Basics - String Formatting

Exercise

Try to fix the code to print out the correct information by changing the string.

script.py

solution.py

```
1/
18 # Convert everything to uppercase
19 print("String in uppercase: %s" % s.upper())
20
21 # Convert everything to lowercase
22 print("String in lowercase: %s" % s.lower())
23
24 # Check how a string starts
25 if s.startswith("Str"):
26     print("String starts with 'Str'. Good!")
27
28 # Check how a string ends
29 if s.endswith("ome!"):
30     print("String ends with 'ome!'. Good!")
31
32 # Split the string into three separate strings,
33 # each containing only a word
```

IPython Shell

```
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs an'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are',
'awesome!']
```

Great work!

Submit

Imaginea 2.6 Learn the Basics - Basic String Operations

Exercise

Change the variables in the first section, so that each if statement resolves as True.

script.py

```
1 # change this code
2 number = 20
3 second_number = 0
4 first_array = [1, 2, 3]
5 second_array = [1, 2]
6
7 if number > 15:
8     print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
```

IPython Shell

```
<script.py> output:
1
2
3
4
5
6

<script.py> output:
1
2
3
4
5
6
```

Great Work!

Solution

Submit

Imaginea 2.7 Learn the Basics - Conditions

Exercise

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

```
219, 390, 984, 592, 236, 105, 942, 941,
4 386, 462, 47, 418, 907, 344, 236, 375, 823,
566, 597, 978, 328, 615, 953, 345,
5 399, 162, 758, 219, 918, 237, 412, 566, 826,
248, 866, 950, 626, 949, 687, 217,
6 815, 67, 104, 58, 512, 24, 892, 894, 767, 553
, 81, 379, 843, 831, 445, 742, 717,
7 958, 609, 842, 451, 688, 753, 854, 685, 93,
857, 440, 380, 126, 721, 328, 753, 470,
8 743, 527
9 ]
10
11 # your code goes here
12 for number in numbers:
13     if number == 237:
14         break
```

IPython Shell

```
592
236
942
386
462
418
344
236
566
978
328
162
758
918
```

Great work!

Solution

Submit

Imaginea 2.7 Learn the Basics - Loops

Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together" [Code block examples](#)
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
5     "More readable code",
6     "Easier code reuse",
7     "Allowing programmers to share and
    connect code together"
8 ]
9
10 # Modify this function to concatenate to each
    benefit - " is a benefit of functions!"
11 def build_sentence(benefit):
12     return benefit + " is a benefit of
    functions!"
13
14 def name_the_benefits_of_functions():
15     list_of_benefits = list_benefits()
16     for benefit in list_of_benefits:
17         print(build_sentence(benefit))
```

IPython Shell

<script.py> output:
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code together
is a benefit of functions!

In [1]: |

Nice work!

Solution

Submit

Imaginea 2.8 Learn the Basics - Functions

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth \$60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth \$10,000.00.

script.py

```
9     return desc_str
10
11 # your code goes here
12 car1 = Vehicle()
13 car1.name = "Fer"
14 car1.color = "red"
15 car1.kind = "convertible"
16 car1.value = 60000.00
17
18 car2 = Vehicle()
19 car2.name = "Jump"
20 car2.color = "blue"
21 car2.kind = "van"
22 car2.value = 10000.00
23
24 # test code
```

IPython Shell

<script.py> output:
Fer is a red convertible worth \$60000.00.
Jump is a blue van worth \$10000.00.

In [1]: |

Great job!

Solution

Submit

Imaginea 2.9 Learn the Basics - Classes and Objects

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py	IPython Shell
<pre>1 phonebook = { 2 "John" : 938477566, 3 "Jack" : 938377264, 4 "Jill" : 947662781 5 } 6 7 # your code goes here 8 phonebook["Jake"] = 938273443 9 del phonebook["Jill"] 10 11 # testing code 12 if "Jake" in phonebook: 13 print("Jake is listed in the phonebook.") 14 15 if "Jill" not in phonebook: 16 print("Jill is not listed in the phonebook.")</pre>	<script.py> output: <pre>Jake is listed in the phonebook. Jill is not listed in the phonebook.</pre> In [1]:
Nice work!	
Solution	Submit ●

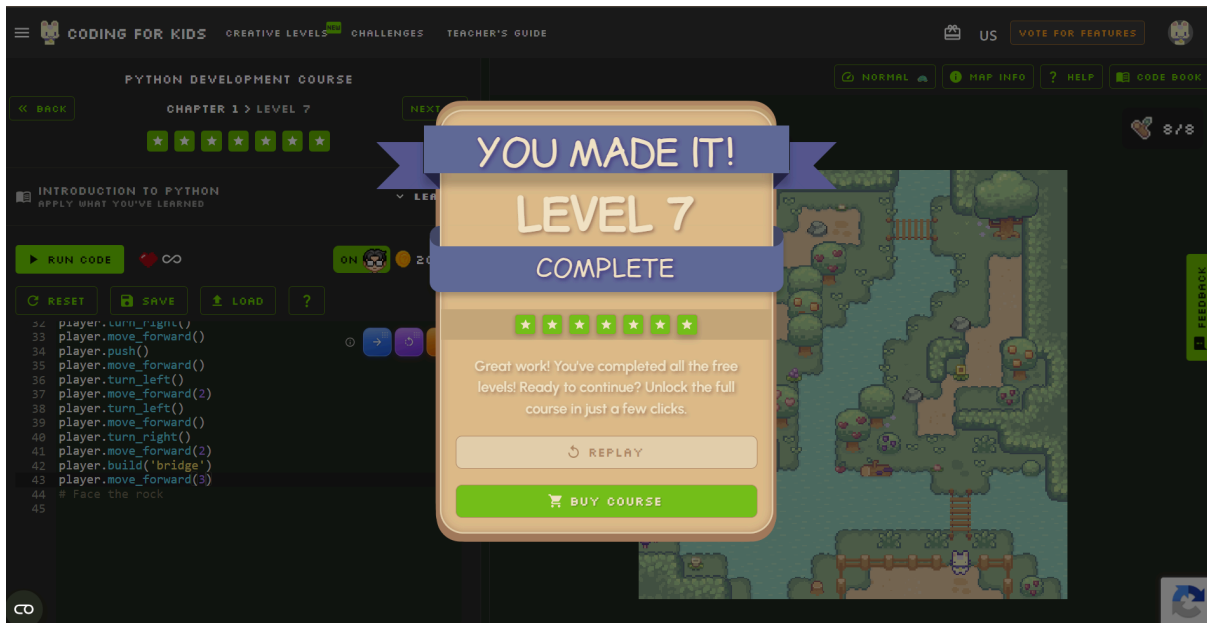
Imaginea 2.10 Learn the Basics - Dictionaries

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py	IPython Shell
<pre>1 import re 2 3 # Your code goes here 4 find_members = [] 5 for member in dir(re): 6 if "find" in member: 7 find_members.append(member) 8 9 print(sorted(find_members))</pre>	<script.py> output: <pre>['findall', 'finditer']</pre> In [1]:
Great work!	
Solution	Submit ●

Imaginea 2.11 Learn the Basics - Modules and Packages



Imaginea 3.1 Coding for Kids - Complete

Exercise

Write a generator function which returns the Fibonacci series. They are calculated using the following formula: The first two numbers of the series is always equal to 1, and each consecutive number returned is the sum of the last two numbers. Hint: Can you use only two variables in the generator function? Remember that assignments can be done simultaneously. The code

script.py

```

1 a = 1
2 b = 2
3 a, b = b, a
4 print(a, b)

```

IPython Shell

```

2 1
In [1]:

```

Run

will simultaneously switch the values of a and b.

script.py

```

1 # fill in this function
2- def fib():
3     a, b = 1, 1
4-     while 1:
5         yield a
6         a, b = b, a + b
7
8 # testing code
9 import types
10- if type(fib()) == types.GeneratorType:
11     print("Good, The fib function is a generator.")
12
13 counter = 0
14- for n in fib():
15     print(n)
16     counter += 1

```

IPython Shell

```

<script.py> output:
Good, The fib function is a generator.
1
1
2
3
5
8
13
21
34
55

```

Good work!

Solution **Submit** **Restart Session**

Imaginea 4.1 Advanced tutorials - Generators

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
2 newlist = [int(x) for x in numbers if x > 0]
3 print(newlist)
```

IPython Shell

<script.py> output:
[34, 44, 68, 44, 12]

In [1]:

Very nice!

Solution

Submit



Imaginea 4.2 Advanced tutorials - List Comprehensions

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1 l = [2,4,7,3,14,19]
2 for i in l:
3     # your code here
4     my_lambda = lambda x : (x % 2) == 1
5     print(my_lambda(i))
```

IPython Shell

<script.py> output:
False
False
True
True
False
True

In [1]:

Nice work!

Solution

Submit



Imaginea 4.3 Advanced tutorials - Lambda functions

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
1 # edit the functions prototype and implementation
2 def foo(a, b, c, *args):
3     return len(args)
4
5 def bar(a, b, c, **kwargs):
6     return kwargs["magicnumber"] == 7
7
8
9 # test code
10 if foo(1, 2, 3, 4) == 1:
11     print("Good.")
12 if foo(1, 2, 3, 4, 5) == 2:
13     print("Better.")
14 if bar(1, 2, 3, magicnumber=6) == False:
15     print("Great.")
16 if bar(1, 2, 3, magicnumber=7) == True:
17     print("Awesome!")
```

IPython Shell

<script.py> output:
Good.
Better.
Great.
Awesome!

In [1]:

Great work!

Solution

Submit

Imaginea 4.4 Advanced tutorials - Multiple Function Arguments

Regular Expressions

Regular Expressions (sometimes shortened to regexp, regex, or re) are a tool for matching patterns in text. In Python, we have the `re` module. The applications for regular expressions are wide-spread, but they are fairly complex, so when contemplating using a regex for a certain task, think about alternatives, and come to regexes as a last resort.

An example regex is `r"^(From|To)Cc.*python-list@python.org"` Now for an explanation: the caret `^` matches text at the beginning of a line. The following group, the part with `(From|To)Cc` means that the line has to start with one of the words that are separated by the pipe `|`. That is called the OR operator, and the regex will match if the line starts with any of the words in the group. The `.*?` means to un-greedily match any number of characters, except the newline `\n` character. The un-greedy part means to match as few repetitions as possible. The `.` character means any non-newline character, the `*` means to repeat 0 or more times, and the `?` character makes it un-greedy.

So, the following lines would be matched by that regex: `From: python-list@python.org To: !aspj<,. python-list@python.org`

A complete reference for the `re` syntax is available at the [python docs](#).

As an example of a "proper" email-matching regex (like the one in the exercise), see [this](#)

script.py

```
1 # Exercise: make a regular expression that will
2 match an email
3 import re
4 def test_email(your_pattern):
5     pattern = re.compile(your_pattern)
6     emails = ["john@example.com", "python-
7 list@python.org", "what?an?ug{}ly@email
8 .com"]
9     for email in emails:
10         if not re.match(pattern, email):
11             print("You failed to match %s" %
12 (email))
13         elif not your_pattern:
14             print("Forgot to enter a pattern!")
15         else:
16             print("Pass")
```

IPython Shell

<script.py> output:
Pass
Pass
Pass
In [1]:

Great work!

Solution

Submit

Imaginea 4.5 Advanced tutorials - Regular Expressions

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1 actor = {"name": "John Cleese", "rank": "awesome"}
2
3 def get_last_name():
4     return actor["name"].split()[1]
5
6 get_last_name()
7 print("All exceptions caught! Good job!")
8 print("The actor's last name is %s" % get_last_name())
```

IPython Shell

<script.py> output:
All exceptions caught! Good job!
The actor's last name is Cleese

In [1]: |

Great work!

Solution

Submit



Imaginea 4.6 Advanced tutorials - Exception Handling

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py

```
1 a = ["Jake", "John", "Eric"]
2 b = ["John", "Jill"]
3
4 A = set(a)
5 B = set(b)
6
7 print(A.difference(B))
```

IPython Shell

<script.py> output:
{'Jake', 'Eric'}

In [1]: |

Nice work!

Solution

Submit



Imaginea 4.7 Advanced tutorials - Sets

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

```
script.py
1 import json
2
3 # fix this function, so it adds the given name
4 # and salary pair to salaries_json, and return it
5 def add_employee(salaries_json, name, salary):
6     salaries = json.loads(salaries_json)
7     salaries[name] = salary
8
9     return json.dumps(salaries)
10
11 # test code
12 salaries = '{"Alfred" : 300, "Jane" : 400 }'
13 new_salaries = add_employee(salaries, "Me", 800)
14 decoded_salaries = json.loads(new_salaries)
15 print(decoded_salaries["Alfred"])
16 print(decoded_salaries["Jane"])
17 print(decoded_salaries["Me"])
```

IPython Shell

<script.py> output:

```
300
400
800
```

In [1]: |

Great work!

Solution Submit

Imaginea 4.8 Advanced tutorials - Serialization

Exercise

Edit the function provided by calling `partial()` and replacing the first three variables in `func()`. Then print with the new partial function using only one input variable so that the output equals 60.

```
script.py
1- #Following is the exercise, function provided:
2 from functools import partial
3 def func(u, v, w, x):
4     return u*4 + v*3 + w*2 + x
5
6
7 #Enter your code here to create and print with your
  partial function
8 p = partial(func,5,6,7)
9 print(p(8))
```

IPython Shell

<script.py> output:

```
60
```

In [1]: |

Good job!

Solution Submit

Imaginea 4.8 Advanced tutorials - Partial functions

Concluzie

În urma realizării acestor sarcini, am avut ocazia să exershez și să înțeleg mai bine limbajul Python, pornind de la lucrurile de bază și ajungând treptat la noțiuni mai avansate. Am lucrat cu variabile, liste, operatori, șiruri de caractere, condiții, cicluri, funcții, clase și obiecte, dicționare, module și

pachete. De asemenea, am rezolvat exerciții pe platformele Codedex și LearnPython, ceea ce m-a ajutat să aplic practic ceea ce am învățat.

Partea cu tutorialele avansate mi-a permis să descopăr și să exersez generatoare, list comprehensions, funcții lambda, argumente multiple, expresii regulate, tratarea excepțiilor, seturi și serializarea. Unele sarcini au fost mai ușoare, iar altele au necesitat mai multă atenție, dar tocmai prin rezolvarea lor am reușit să înțeleg mai bine cum funcționează Python.

În general, consider că lucrările realizate au fost utile și interesante, deoarece nu am învățat doar partea teoretică, ci am avut posibilitatea să scriu și să testez cod. Aceste exerciții m-au ajutat să capăt mai multă încredere în lucrul cu Python și să fiu mai pregătit pentru sarcini mai complexe pe viitor.